Is Hyper-Parameter Optimization Different for SE?

Yes. And here's the math to prove it.

Rahul Yedida (NCSU) Tim Menzies (NCSU)

NC State University, USA
http://timm.fyi

TSE, 51(6), 2025, https://ieeexplore.ieee.org/document/10919477
SLIDES: http://timm.fyi/26smooth.pdf

Apr 17, 2026

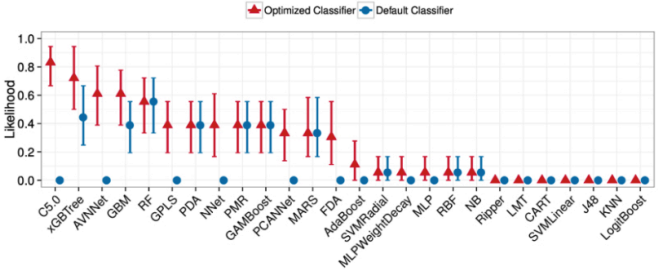
Message

- AI for SE is **different** to standard AI.
- Previously:
  - DRR: data reduction ratio: SE data tends to "flatten" far more than data from other sources.
  - **https://arxiv.org/abs/2503.21086**
  - Above the "Andre threshold", astonishly simple methods work well
    - For defect prediction
    - For Github issue **lifetime prediction**
    - For benchmakrs from NeurIPS'20
    - For detecting **fasle alarms**in static code warnigns.
- Here:
  - Maths defining **when**to be simpler
  - Fast sampling to apply that maths
    - e.g. just run a deep learning for a few **epochs**

Mathematics of "easy"

- SE data has "smoother" **boundaries** between classes.
- Compared to AI data, magnitude of second derivative of loss function in SE data is much **smaller**.
- For SE data, better HPO result can be achieved using **AI designed for SE**.

Motivation: Off-the-Shelf AI for SE Data



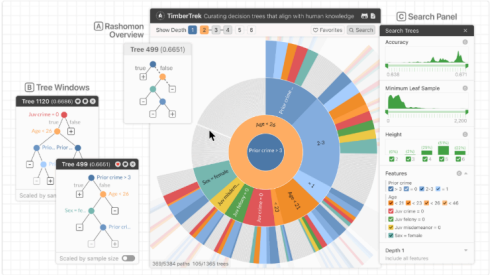
[Tantithamthavorn et al., ICSE'16]

Most SE research uses AI defaults without audit.

- HPO = hyperparameter optimization. Black art of selecting tuning parameters.
- HPO is applied to defect prediction and issue triage.
- Current tools (NSGA-II, MOEA/D, SMAC3, BOHB, DEHB, etc) target general AI data.
- Question: Is SE data different? Does it need different kinds of learning?

Rashomon Volumes

Given so many good models, epistemically dishonest to reason about "the" model



(a) Rashomon set of the 1,365 best sparse decision trees for the COMPAS dataset, generated by TreeFARMS and TimberTrek (figure from Wang et al. 20

Amazing Things Come From Having Many Good Models

Cynthia Rudin [1] * Chudi Zhong [1] Lesia Semenova [1] Margo Seltzer [2] Ronald Parr [1] Jiachang Liu [1] Srikar Katta [1] Jon Donnelly [1] Harry Chen [1] Zachery Boner [1]

Abstract

The *Rashomon Effect*, coined by Leo Breiman, describes the phenomenon that there exist many equally good predictive models for the same dataset. This phenomenon happens for many real datasets and when it does, it sparks both magic and consternation, but mostly magic. In light of the Rashomon Effect, this perspective piece proposes reshaping the way we think about machine learning, particularly for tabular data problems in the nondeterministic (noisy) setting. We address how the Rashomon Effect impacts (1) the existence of simple-yet-accurate models, (2) flexibility to address user preferences, such as fairness and monotonicity, without losing pre... (3) uncertainty in prediction... nations, (4) relat... gorith...

think of the Rashomon Ef... from getting a...

The Rashomon Effect for Visualizing High-Dimensional Data

Yiyang Sun* Haiyang Huang*+ Gaurav Rajesh Parikh* Cynthia Rudin

Efficient Rashomon Set Approximation for D... Trees

Zakk Heile
Department of Computer Science
Duke University
Durham, NC 27708
zakk.heile@duke.edu

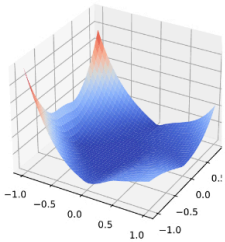
Hayden McTavish
Department of Computer Science
Duke University
Durham, NC 27708
hayden.mctavish@duke.edu

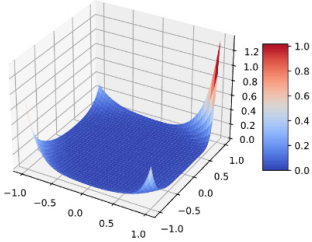
Varun Babbar
Department of Compute...
Duke Universit...
Durham, NC 27...
varun.babbar@du...

Cynthia Ru...
Department of Com...
Duke Univ...
Durham, NC...
cynthia@cs...

The Observation: SE Loss Landscapes are Smoother

Claim: SE model loss landscapes are exploitably smooth.

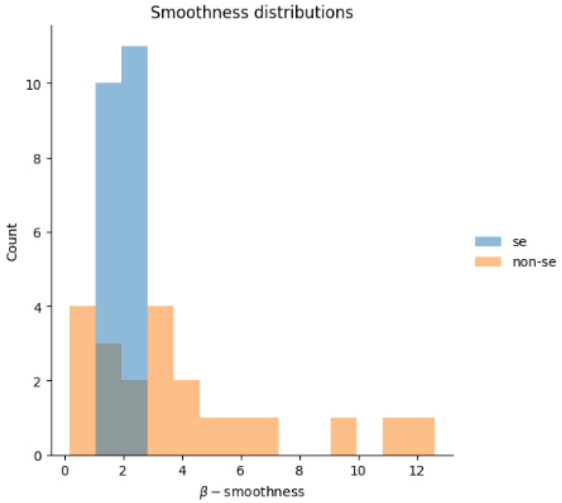




What "smooth" means (formally)

f is β-smooth if: $\frac{\|\nabla f(y) - \nabla f(x)\|}{\|y - x\|} \leq \beta$.

- SE data: mean β-smoothness ≈ 2.
- Standard AI (UCI) data: mean β-smoothness ≈ 5.


Smoothness distributions
Count
0
2
4
6
8
10
0
2
4
6
8
10
12
$\beta -$ smoothness
se
non-se

Why SE Data is Smooth: The Naturalness Argument

- Human code is surprisingly repetitive [Hindle et al. ICSE'12]
- Maintainability forces repeated use of idioms and patterns.
- Repetition $\Rightarrow$ minimal variation between clusters.
- Minimal variation $\Rightarrow$ flat loss landscapes.

The core insight

Maintainable code produces data with flat loss landscapes.

Effects of HPO

Smoothing the landscape.

Experiments with Hyper-parameters (not an exhaustive list)

Pre-processing (Basic)

Normalization: $x \rightarrow \frac{x}{\|x\|}$

Standardization: $x \rightarrow \frac{x-\mu_x}{\sigma_x}$

Min-max scaling: $x \rightarrow \frac{x-\min x}{\max x-\min x}$

Max-abs scaling: $x \rightarrow \frac{x}{\max |x|}$

Robust scaling: $x \rightarrow \frac{x-P_{50}(x)}{P_{75}(x)-P_{25}(x)}$

Pre-processing (Advanced)

SMOTE: Synthetic minority samples via interpolation.

Fuzzy sampling: Allocates samples across layers ($\frac{1}{2^i}$) to rebalance.

Semi-supervised labeling: Label deletion and k-NN rebuild.

Neural Networks

Feedforward: 3–20 layers, 2–6 units/layer.

Deep Learners: More control parameters, e.g., topology, activation functions.

Traditional Classifiers

Neighbor classifiers: (a) Distance measure, (b) neighbor count (k), (c) combination method, (d) pre-clustering.

Random forests: Number of trees, features per tree, polling method (majority or weighted).

Logistic regression: (a) Regularization type (L1, L2, elastic net), (b) strength ($C \in \{0.1, 1, 10\}$).

The Archive: 23 Datasets across 30 Years

Dataset	#Trn	#Tst	Feat
Defect prediction			
camel	1819	442	20
ivy	352	352	20
log4j	244	205	20
synapse	379	256	20
velocity	410	229	20
xalan	2411	909	20
Static code warnings			
ant	28	7	329
cassandra	13	4	265
commons	17	5	127
derby	366	92	292
jmeter	14	4	287
lucene-solr	24	6	278
tomcat	147	37	284

Dataset	#Trn	#Tst	Feat
Issue Lifetime Prediction			
Eclipse	44545	25459	12
Firefox	44800	25201	12
Chromium	44801	25200	12
Bayesmark (from NuerIPS'20)			
breast	455	114	30
digits	1438	359	64
iris	120	30	4
wine	142	36	13
diabetes	354	88	10

The Algorithm: SMOOTHIE

Sample $N_1 = 30$ configs

↓

Compute smoothness β

↓

Keep top $N_2 = 5$ configs

↓

Run those N_2 fully

For NNs: $\beta \propto \|W^{[L]}\|$ after 1 epoch [**?**].

Finding 1 (RQ1): SMOOTHIE Wins on SE Data

- **Static warnings:** SMOOTHIE wins 4, ties 3.
- **Issue lifetime:** Ties best in all 3 datasets.
- **Defect prediction:** All methods tie (low complexity).

RQ1 Answer

SMOOTHIE outperforms or matches SOTA on SE-specific data.

Finding 2 (RQ2): SMOOTHIE is Much Faster

- Computing β (Get-Smoothness) is negligible.
- End-to-end, SMOOTHIE is 4.7× faster than BOHB.
- End-to-end, SMOOTHIE is 8.3× faster than DEHB.

RQ2 Answer

SMOOTHIE achieves SOTA accuracy 4–8× faster.

The Why: Why Does SMOOTHIE Win on Flat Landscapes?

- BOHB needs contrast (bumpy landscapes) to explore.
- Flat loss landscapes generalize better to test data [**?**].
- Starting near $\beta \approx 2$, flattening hits the sweet spot.

Key design trade-off

SMOOTHIE biases for generalization, not training performance.

Takeaways: What This Means for SE Research

1. Stop assuming general AI tools transfer to SE.
2. Compute β as a diagnostic: if < 4, use SMOOTHIE.
3. SE data has unique geometry requiring unique algorithms.